AI System Architecture Proposal: Produce Recognition for Grocery Self-Checkout

Ryan Schader
ECE 6/5540

September 12, 2026

1 Problem Statement

The users are grocery stores and their customers. The system receives a visual input (a picture of produce), a scale weight, and the store's currently stocked PLUs. It produces a suggested produce name, PLU/SKU, and price calculation. This would save grocery stores and customers time: instead of manually inputting what type of produce they are getting, customers can place the produce on a scale with a camera looking at it. It will then be automatically identified.

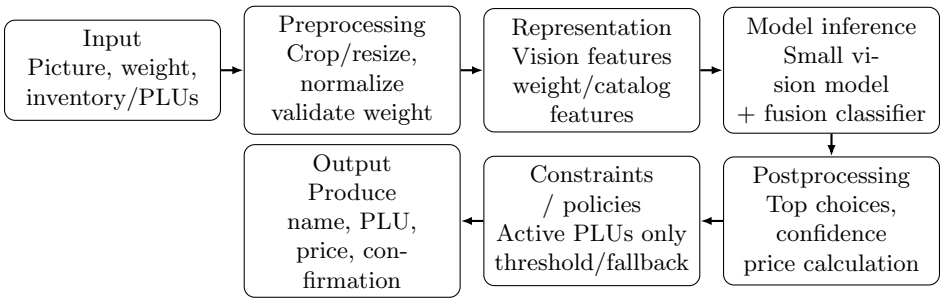
The problem is limited to recognizing common loose produce at a self-checkout station. An AI model is appropriate because it can use visual details such as color, shape, and texture to identify produce, while weight and the store inventory help distinguish similar items.

2 Chosen Modality and Task

The system is a multimodal vision-and-weight system. The main task is closed-set produce classification: choose from the produce items that the store currently sells. A lightweight detector can first check that produce is visible, and the final system will combine the vision result with the required scale weight, price, current inventory, season, and currently stocked PLUs.

This modality is appropriate because the camera provides the main evidence for recognition, while the required weight input and other structured information help make the decision more accurate without requiring an unbounded data source.

3 Proposed System Architecture



A camera takes a picture of the produce, and the station reads the scale weight and active store inventory.

The image is cropped, resized, and normalized; blurry, blocked, or empty images and unstable weights are rejected. The image becomes vision features, while weight, price, season, and stocked PLUs become small numerical or categorical features. A small vision model identifies likely produce, and a small fusion classifier always combines the vision embedding with the scale weight, price/store inventory, season, and currently stocked PLUs. The system keeps the top choices, checks confidence, and calculates a price after selection. It only selects active PLUs. If confidence is low or information conflicts, it asks the customer to choose from a short list or re-scan. The customer then sees the produce name, PLU/SKU, weight, and price and confirms the selection.

4 Initial Model Selection

I would benchmark MobileNetV3/V4, EfficientNet-Lite, YOLO11n-cls or YOLO11n detect, and possibly a small VLM to see which performs best. The initial candidate is MobileNetV3-Large, a compact model of about 5 million parameters. These models would be best because they can run on edge hardware and, with some tuning and the right architecture, be very accurate.

MobileNetV3 fits the architecture because it is small enough for low-memory, low-compute checkout hardware and can provide the vision features for the fusion classifier. A YOLO detector may be useful if hands, multiple items, or other objects are in the image. A much larger model likely will not fit limited hardware or meet the speed requirement; a much smaller model may not accurately separate similar produce.

5 Architectural Constraints

- The model needs to run on limited edge hardware, so a compact model and possible quantization are needed.
- The result should appear in about one second after the scale settles, which favors local inference instead of a remote service.
- Many checkout lanes make cloud inference expensive and network dependence risky. The core system should work locally, with only model and catalog updates needing a network.
- A wrong produce classification can charge the wrong price. Confidence thresholds and customer confirmation are required before completing the transaction.

6 Expected Failure Modes

Failure mode	Type	What must change
Wrong produce classified, especially similar items.	Model failure	Add labeled images, balance classes, improve augmentation, and tune the confidence threshold/fusion classifier.
Produce is not recognized because a hand or other object is in the way.	Architectural failure	Add image-quality checks, improve camera/lighting placement, ask for a re-scan, or add a detector before classification.
The predicted produce is not currently stocked.	Architectural failure	Keep a validated local inventory/PLU list and restrict output to active items; provide manual lookup.

7 Semester Roadmap

During the prototype stage, I will collect a small bounded produce dataset, probably from Hugging Face, build a baseline classifier, and measure top-1/top-3 accuracy, latency, and manual-selection rate. During optimization, I will benchmark the candidate models, add lighting and occlusion augmentation, tune the fusion step, and test quantization. During deployment, I will run the selected model with the scale interface and local inventory cache on edge hardware and test offline operation and fallbacks. During evaluation, I will measure accuracy by class, confusion between similar produce, end-to-end latency, correction rate, and price-selection errors.